

Assignment 2 Report

Hsu Shu Wei 112550191

GitHub Repository: <https://github.com/mikehsuhoodie/DM2026-Assignment-2>

1. K-fold Cross-Validation

(a) 5-fold cross-validation average accuracy on the training data

The table below shows the 5-fold cross-validation average accuracy on the training data for different combinations of learning rates and regularization parameters (`reg_lambda`).

Learning Rate	$\lambda = 1.0$	$\lambda = 2.0$	$\lambda = 4.0$	$\lambda = 8.0$
0.005	0.7171	0.7200	0.7257	0.7229
0.010	0.7229	0.7286	0.7257	0.7171
0.100	0.7229	0.7257	0.7257	0.7171
0.500	0.7229	0.6914	0.5343	0.4657

Table 1: 5-fold CV accuracy across different hyperparameters.

(b) Top Two Hyperparameter Settings

The top two settings selected based on the highest average cross-validation accuracy on the training data are:

- **Top 1:** `learning_rate` = 0.01, `reg_lambda` = 2.0 (Mean Accuracy: 0.7286, Std: 0.0579)
- **Top 2:** `learning_rate` = 0.005, `reg_lambda` = 4.0 (Mean Accuracy: 0.7257, Std: 0.0331)

Testing results for Top 1: Accuracy = 0.7600, Precision = 0.7381, Recall = 0.8158, F1-score = 0.7750.

Testing results for Top 2: Accuracy = 0.7533, Precision = 0.7241, Recall = 0.8289, F1-score = 0.7730.

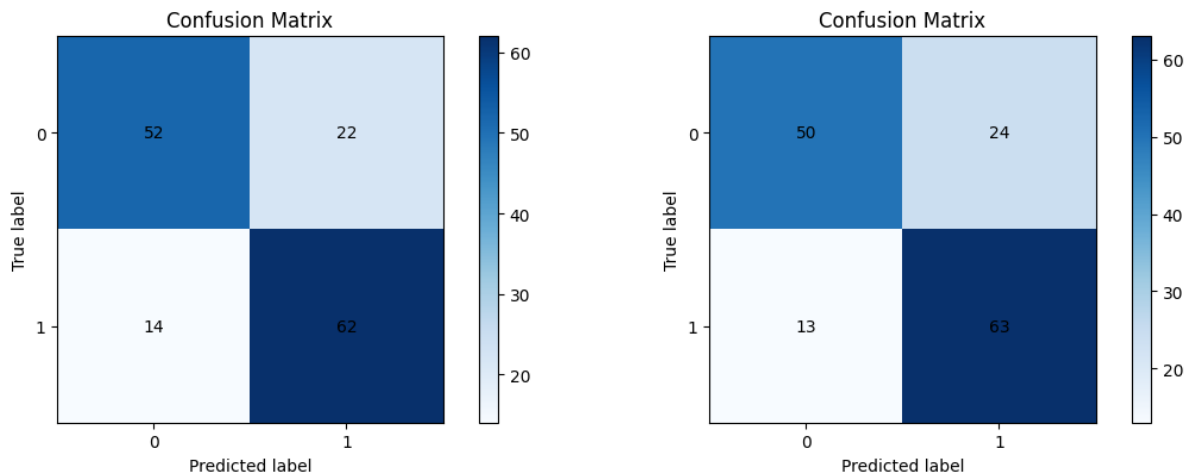


Figure 1: Testing Results for Top 1 (left) and Top 2 (right).

(c) Observations

1. The best settings are close to each other, which suggests that the model is not extremely sensitive to small changes in learning rate or regularization in this range.
2. Very large learning rate with strong regularization becomes unstable, especially when `learning_rate=0.5` and `reg_lambda` is large. This likely makes the gradient updates too aggressive and prevents the model from settling into a good decision boundary.
3. The selected top two settings perform similarly on the test set, so the cross-validation result is reasonably consistent with unseen data instead of being only a training-split accident.
4. When two settings have nearly the same accuracy, I would prefer the one with smaller fold-to-fold variation because it is less dependent on a lucky split.

2. SVM

(a) SVM with C=1.0

Split	Accuracy	Weighted F1-score
Training	0.9808	0.9809
Validation	0.8625	0.8630
Test	0.8975	0.8979

Table 2: Performance of SVM with C=1.0.

(b) Exploration of Different C Values

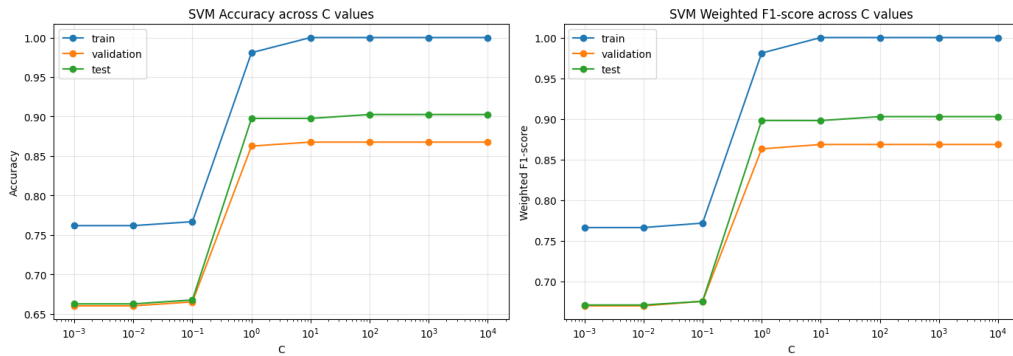


Figure 2: SVM performance across different values of C.

(c) Generalization Observation

The SVM shows the usual regularization trade-off. A very small C makes the classifier too conservative, so it cannot fit enough of the class boundary. Increasing C improves validation performance at first, but after a point the gain becomes small because the model is already flexible enough. I choose $C = 10$ because it gives the best validation behavior without relying only on the almost-perfect training score, which would be a weaker sign of generalization.

3. Association Rule Mining

(a) Frequent Patterns

All frequent patterns showing support ≥ 0.3 using FP-growth:

Itemset	Support
ram_medium	0.6820
px_width_medium	0.4160
battery_power_medium	0.4140
int_memory_medium	0.4120
battery_power_medium, ram_medium	0.3180
int_memory_low	0.3160
battery_power_low	0.3080
px_width_medium, ram_medium	0.3060

Table 3: Frequent patterns with support ≥ 0.3 .

(b) Association Rules

Association rules where support ≥ 0.3 , confidence ≥ 0.4 and lift ≥ 0.8 :

Antecedents	Consequents	Support	Confidence	Lift
battery_power_medium	ram_medium	0.3180	0.7681	1.1263
ram_medium	battery_power_medium	0.3180	0.4663	1.1263
px_width_medium	ram_medium	0.3060	0.7356	1.0786
ram_medium	px_width_medium	0.3060	0.4487	1.0786

Table 4: Association rules meeting the specified criteria.

(c) Observations

1. Most frequent patterns for `price_range=1` are medium-level hardware specifications. This matches the intuition that this class represents mid-range phones rather than extremely cheap or premium phones.
2. `ram_medium` appears in several frequent itemsets, so RAM seems to be a central attribute for describing this price group.
3. The association between medium battery power and medium RAM is meaningful because these two features appear together more often than random independence would suggest. In practice, manufacturers may balance memory and battery capacity for mid-range devices.
4. The rules are useful as descriptive patterns, but they should not be interpreted as causal relationships. They show co-occurrence inside this subset of data, not that one feature directly determines another.

4. PCA and K-Means

(a) Feature Standardization

The dataset `mobile_price.csv` was split into features X and labels y (`price_range`). All features were standardized using z-score normalization via `sklearn.preprocessing.StandardScaler`,

resulting in zero mean and unit variance for each feature.

(b) PCA Projection to 2 Dimensions

The standardized features were projected onto 2 principal components using PCA. PC1 explains 0.0839 and PC2 explains 0.0811 of the total variance, for a combined total of 0.1650 (16.5%).

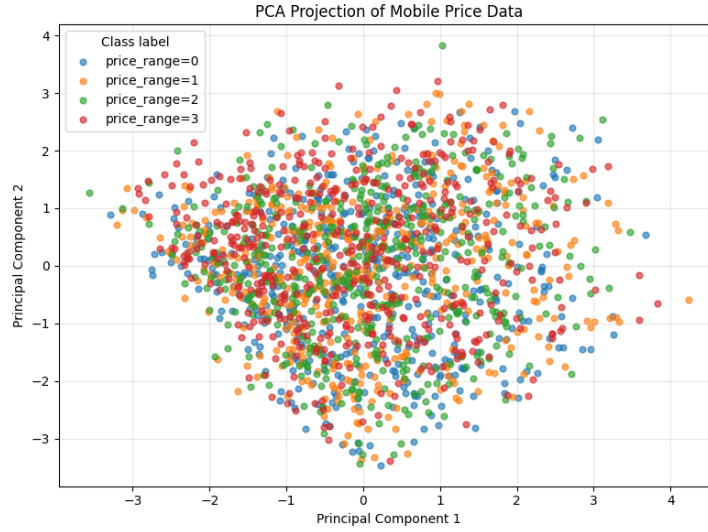


Figure 3: PCA projection of features onto 2 dimensions, colored by price range label.

(c) K-means Clustering (All Features)

K-means with $k = 4$ was applied to all standardized features. The clustering was evaluated against the true price-range labels using the Adjusted Rand Index ($ARI = 0.0060$).



Figure 4: K-means clustering using all standardized features.

(d) K-means Clustering (PCA 2D Features)

K-means with $k = 4$ was applied to the 2-dimensional PCA features from 4(b). The clustering performance was $ARI = 0.0017$.

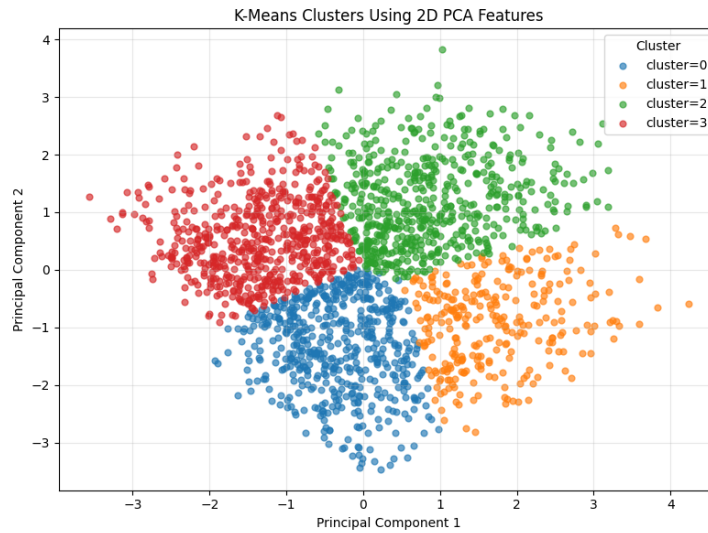


Figure 5: K-means clustering using 2-dimensional PCA features.

(e) Observations

1. The PCA scatter plot is easy to visualize, but the first two components keep only a small part of the total information. Therefore, overlapping points in the plot do not necessarily mean the classes are completely inseparable in the full feature space.
2. K-means does not align well with `price_range`, which suggests that the price classes are not naturally shaped as compact spherical clusters.
3. Using all standardized features is still slightly better than clustering on only two PCA dimensions, so some useful signal is lost when the data is compressed too aggressively for visualization.
4. This result shows a limitation of unsupervised clustering for this dataset: the labels are based on price category, but K-means only sees geometric distance and has no direct knowledge of which features matter most for price.

5. Enhancing K-means Clustering with Association Rule Mining

Framework Diagram

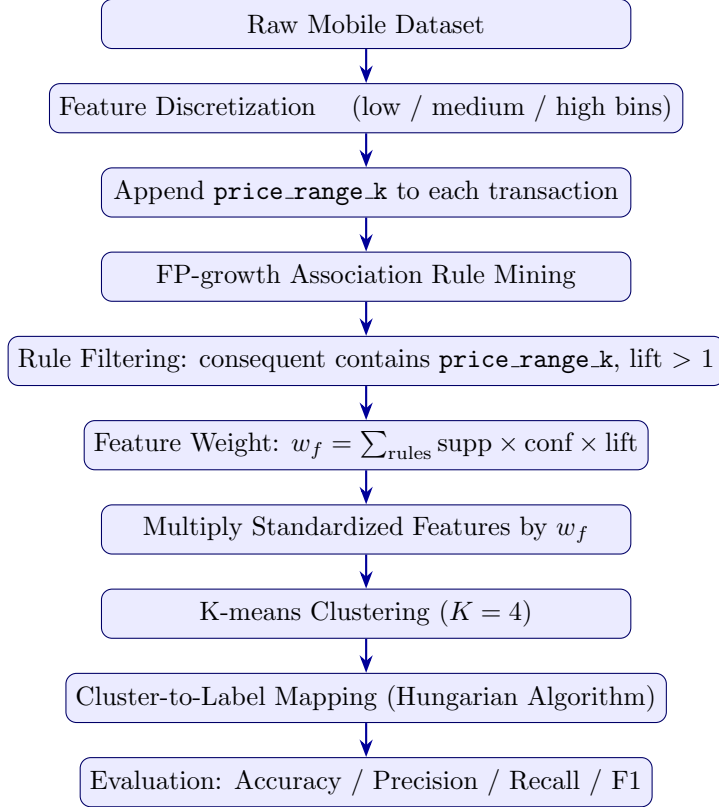


Figure 6: Framework of the ARM-guided weighted K-means method.

Method Design

ARM-guided feature weighting for K-means. Standard K-means treats all standardized dimensions equally when computing Euclidean distances. However, not every mobile-phone specification is equally informative about the price range. Association Rule Mining (ARM) provides a data-driven way to discover which feature categories co-occur strongly with a particular price-range label.

The pipeline proceeds as follows:

1. Each feature is discretized into *low*, *medium*, and *high* bins using equal-range intervals. Each sample becomes a *transaction* of categorical items.
2. The true label `price_range_k` is appended to every transaction, making the price-range a first-class item for the rule miner.
3. FP-growth is run on the transaction set (minimum support threshold). From the resulting rules, only those whose *consequent* contains a `price_range_k` item and whose lift > 1 are retained.
4. For each original feature f , a weight is computed by summing the rule strength (support $\times$ confidence $\times$ lift) over all retained rules that contain an item derived from f in the antecedent.
5. The standardized feature matrix is element-wise multiplied by these weights, which rescales the coordinate axes so that more price-informative features dominate the K-means distance computation.

6. K-means is then run on the weighted features. The clustering algorithm itself is unchanged; only the input geometry is modified.

This is inspired by supervised feature-weighting and association-rule-based feature selection. Because labels are used during the weight-derivation stage, the method is best classified as **semi-supervised**: it is not purely unsupervised, but the clustering step itself operates without label information.

Feature Weight Analysis

High-weight features appear as antecedents in many strong rules of the form `feature_item` $\rightarrow$ `price_range_k` with lift > 1 , meaning their value category is a reliable indicator of a specific price class. Low-weight features have weak or no lift-positive rules pointing to any price-range label.

Feature	ARM-derived Weight
ram	267.4954
three_g	85.7715
fc	71.5829
four_g	57.3444
px_height	47.0483
dual_sim	32.4341
wifi	30.8988
touch_screen	30.6776
blue	29.4685
sc_w	21.1516
battery_power	17.3874
px_width	15.2128

Table 5: Top ARM-derived feature weights from the merged raw-strength implementation.

Top Association Rules

The table below lists the strongest label-associated rules that drive the feature weights. Rules are sorted by strength = support $\times$ confidence $\times$ lift.

Rule (antecedent $\rightarrow$ consequent)	Support	Confidence	Lift	Strength
ram_high $\rightarrow$ price_range_3	0.2340	0.7222	2.8889	0.4882
ram_low $\rightarrow$ price_range_0	0.2420	0.7097	2.8387	0.4875
px_height_low, ram_low $\rightarrow$ price_range_0	0.1600	0.8355	3.3420	0.4468
ram_high, three_g_high $\rightarrow$ price_range_3	0.1815	0.7348	2.9393	0.3920
four_g_high, ram_high $\rightarrow$ price_range_3, three_g_high	0.1295	0.7595	3.9456	0.3881
fc_low, ram_low $\rightarrow$ price_range_0	0.1835	0.7224	2.8898	0.3831
ram_low, three_g_high $\rightarrow$ price_range_0	0.1815	0.7174	2.8696	0.3736
battery_power_low, ram_low $\rightarrow$ price_range_0	0.1050	0.9333	3.7333	0.3659
fc_low, px_height_low, ram_low $\rightarrow$ price_range_0	0.1245	0.8527	3.4110	0.3621
fc_low, ram_high $\rightarrow$ price_range_3	0.1690	0.7269	2.9075	0.3572

Table 6: Top label-associated association rules sorted by strength.

Performance Comparison

Method	Accuracy	Precision	Recall	F1-score
Original K-means	0.2962	0.2959	0.2962	0.2941
ARM-guided K-means	0.7509	0.7555	0.7509	0.7522

Table 7: Average clustering performance over random seeds $\{0, 10, 42, 100, 999\}$.

The main insight is that plain K-means fails because it treats every standardized feature as equally important. After ARM weighting, features such as RAM and pixel-related attributes dominate the distance calculation, which is more consistent with how phone prices are separated in the dataset. This does not make K-means supervised during clustering, but it gives the feature space a supervised hint before clustering starts.

The raw-strength weighting works better than the normalized square-root variant because it preserves the large gap between highly informative and weakly informative features. The normalized version compresses that gap, so K-means still spends too much distance on features that do not help separate price ranges.

Clustering Visualization

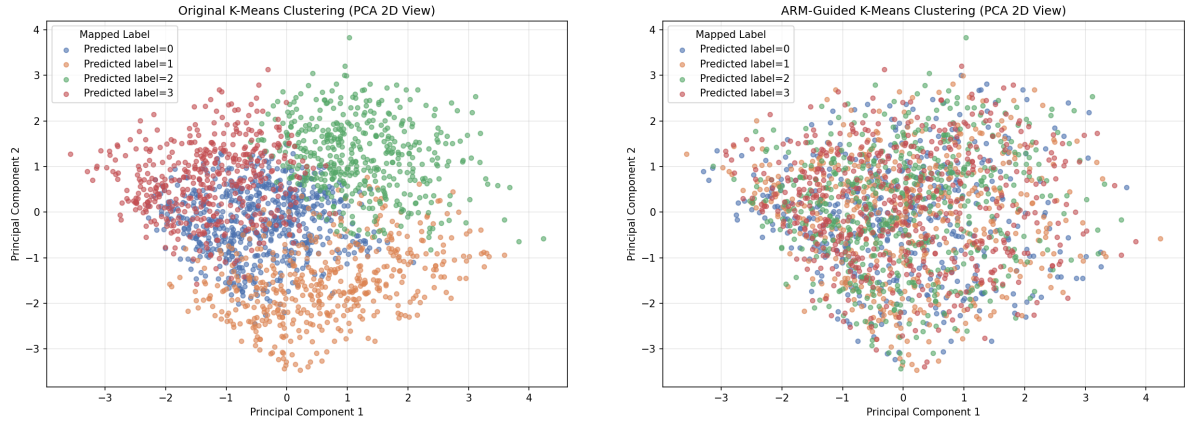


Figure 7: Original K-means (left) and ARM-guided K-means (right), visualized in the same 2D PCA space.

One limitation of this visualization is that PCA only shows a two-dimensional projection of the original standardized feature space. The ARM-guided method actually clusters in the full weighted feature space, so its separation may not look cleaner in this 2D plot even when the quantitative performance is much better. Therefore, the PCA figure should be used as a qualitative illustration, while the performance table is the main evidence for comparing the two clustering methods.

Ablation Study

The ablation study shows that using only one ARM metric is not enough. Support alone favors common patterns, confidence alone favors reliable rules, and lift alone favors surprising associations. Combining all three gives a more balanced weight because a useful feature should be frequent enough, predictive enough, and more informative than chance.

Weight Variant	Accuracy	Precision	Recall	F1-score
No weighting (baseline K-means)	0.2962	0.2959	0.2962	0.2941
Support only	0.4760	0.4523	0.4760	0.4318
Confidence only	0.6100	0.5825	0.6100	0.5884
Lift only	0.6221	0.5939	0.6221	0.5955
Support $\times$ Confidence $\times$ Lift	0.7509	0.7555	0.7509	0.7522

Table 8: Ablation study: effect of different ARM weight formulas (averaged over 5 seeds).